An Empirical Study of Kubernetes HPA Behavior for Stateful WebSocket Workloads

Abhash Behera · Aditya Hota · Suprit Kumar Naik · Ritesh Kumar Singh · Rakesh Ranjan Kumar · Binita Kumari

Abstract Kubernetes' Horizontal Pod Autoscaler (HPA) scales deployments using a proportional control law built around CPU utilization. That works fine for stateless, request-driven services, but it runs into trouble with stateful, persistent-connection protocols such as WebSocket, where session state lives on a specific pod whether or not the connection happens to be busy at that moment. Kill the pod and every connection it was holding goes with it. We ran four controlled experiments to pin down exactly how this plays out. Experiment A gives us a correct-behavior baseline under near-ideal conditions. Experiment B1 puts HPA through a realistic HIGH/LOW activity cycle and shows it over-provisions badly and recovers slowly. Experiment B2-Instrumented measures what actually happens during a scale-down event: a large reconnection storm that hits right as pods are terminated and throws the session population into disarray. Experiment B3 then shows that when clients don't retry on their own, these events destroy sessions permanently, in a staircase pattern, with nothing bringing the numbers back. Across all four experiments, the failure traces back to how HPA's CPU-based design interacts with persistent-connection workloads rather than to any single misconfigured parameter. CPU simply isn't a good stand-in for connection demand, and tuning parameters alone doesn't fix it. These findings feed directly into the connection-aware autoscaling approach explored in subsequent work.

Keywords Kubernetes · Horizontal Pod Autoscaler · WebSocket · Stateful Workloads · Autoscaling · Reconnection Storm · Cloud Computing · Performance Evaluations

Corresponding author. Email: rakeshranjan@cgu-odisha.ac.in

Department of Computer Science and Engineering,
C V Raman Global University,
Bhubaneswar 752054, Odisha, India

Emails:
2301020213@cgu-odisha.ac.in,
2301020486@cgu-odisha.ac.in,
2301020457@cgu-odisha.ac.in,
2301020845@cgu-odisha.ac.in,
rakeshranjan@cgu-odisha.ac.in,
binita.kumari@cgu-odisha.ac.in

1 Introduction

Kubernetes has become the default choice for container orchestration in the cloud [12, 22], and a big part of why people like it is the Horizontal Pod Autoscaler (HPA), which watches resource metrics on a fixed cycle and adjusts replica counts through a proportional control law [15, 23, 4]. For ordinary HTTP services, where CPU load tracks request volume reasonably well, this works about as expected [6, 20].

The problem shows up with WebSocket [9] and similar protocols: a connection that stays open for minutes, sometimes hours, doing full-duplex work the whole time. Persistent WebSocket connections, MQTT, collaborative editors, multiplayer games, IoT gateways, and long-lived streaming pipelines are all increasingly common in production, so getting autoscaling right for them actually matters. For these workloads, CPU isn't really the resource that counts - what counts is the open connection itself, since the session state sits on one particular pod. Terminate that pod and every connection it held drops at the same instant, so every affected client tries to reconnect at once. We call this a reconnection storm.

Here's the core issue in one sentence: CPU usage tells you nothing about how many connections a pod

is actually holding. A pod can be quietly serving 800 healthy WebSocket sessions while barely touching the CPU, and from HPA's point of view that looks identical to a pod doing nothing at all - so it gets removed either way.

None of this is a brand-new observation. Dickel et al. [7] already pointed out that stateful protocols don't play well with HPA. What's missing from that prior work is any real measurement of how bad the problem gets, or a concrete reconnection-storm rate. That's the gap we're trying to close here, and our contributions break down as follows:

1. **Four experiments that build on each other.** A, B1, B2-Instrumented, and B3 run in sequence, moving from over-provisioning, to a measured reconnection storm, to permanent connection loss.
2. **An actual measured storm rate.** With Prometheus instrumentation in place, we measure a mean peak of 457 connections/second triggered by one single HPA scale-down decision. Earlier work only described this qualitatively; we put a number on it.
3. **A step-by-step account of the mechanism.** Experiment B3 walks through the sequence: connections go idle, CPU drops toward zero, HPA scales down in response, and the sessions on the terminated pod are gone for good. All of this happens using nothing more exotic than default CPU-based HPA behavior.
4. **The stabilization window doesn't save you.** HPA's stabilization window smooths out CPU-derived replica recommendations - it has nothing to do with connection counts. Once CPU sits near zero for long enough, every value going into that window is `minReplicas`, so no matter how you size the window, the buffered output still ends up at `minReplicas`.
5. **Concrete numbers to benchmark against.** Every failure mode we describe comes with a number attached - 233% replica over-allocation, a 457 conn/s storm peak, 18.8% mean connection loss - which should give future autoscaling work something to compare against.

As far as we can tell, this is one of the first empirical studies to systematically characterize Kubernetes HPA behavior under persistent WebSocket workloads in a way that actually quantifies reconnection storms, over-provisioning, and permanent session destruction, all within one unified experimental setup.

The rest of the paper goes like this. Section 2 covers related work, Section 3 gives a quick recap of the HPA control loop, Section 4 walks through the four experiments in detail, Section 5 gets into why this happens and what it means in practice, Section 6 covers what we couldn't test, and Section 7 wraps things up.

2 Related Work

2.1 Kubernetes HPA Architecture and Behavior

Nguyen et al. [23] analyzed the HPA feedback loop and scrape intervals, but their work assumes stateless HTTP services throughout; we instead look at what happens under persistent WebSocket workloads, where CPU utilization stops reflecting actual service demand. Recent studies characterizing Kubernetes resource management and autoscaling behavior similarly highlight the complexity of these operations [21]. Casalicchio [5] showed that choosing relative versus absolute metrics changes scaling stability, but again, their findings sit entirely within HTTP/CPU workloads and don't touch connection-holding protocols. Lorido-Botrán et al. [20] lay out theoretical foundations for proportional controllers and cloud resource adaptation more broadly; we lean on that framing to show where these controllers actually break down for stateful sessions.

2.2 Custom and Application-Level Metrics for HPA

Kubernetes does support custom metrics through the Prometheus Adapter [18], but the existing literature mostly skips over connection state entirely. Qu et al. [27] and Rossi et al. [28] propose enhanced autoscaling mechanisms, but they remain primarily focused on improving scaling efficiency rather than preserving persistent connection state. More recent predictive autoscaling schemes also rely on workload forecasting to improve HPA responsiveness [1], but they continue assuming workload intensity is adequately represented through aggregate resource metrics. Recent industrial deployments such as AHPA [30] further improve scaling responsiveness using prediction models, but continue optimizing resource allocation rather than preserving application-level connection state. Our study targets exactly the persistent-session problem these approaches miss, and we show that even with a correct connection-count metric, HPA's stabilization window still buffers metric-derived recommendations instead of connection-derived ones. That's a structural limitation we dig into in Section 5.

2.3 Stateful Workloads and Protocol-Aware Autoscaling

Dickel et al. [7] flagged stateful autoscaling as an open problem for WebSocket/MQTT gateways and suggested tracking active connections through Prometheus, but they didn't validate this quantitatively. We take that further and empirically characterize the actual failure modes, putting numbers on the operational consequences these architectural gaps produce, which gives future protocol-aware autoscaling work a baseline to work from.

2.4 Graceful Connection Draining

Production environments generally lean on `preStop` hooks and `terminationGracePeriodSeconds` [14] so stateless pods can drain in-flight requests before shutting down. That mechanism just doesn't work for persistent sessions. A typical HTTP request resolves in milliseconds; a WebSocket session can run for hours, so a standard grace period is essentially irrelevant at that timescale. Experiment B3 (Section 4.4) puts a number on exactly how much permanent connection loss results when these standard mechanisms fail to help.

2.5 Research Gap

Most existing studies evaluate HPA using stateless, request-driven workloads, or they propose custom metrics without ever quantifying what CPU-driven scaling actually costs under persistent connection workloads. Because of that, the magnitude, reproducibility, and root cause of these failures remain poorly characterized. This paper tries to close that gap by pinning down the exact failure mechanisms with connection-second precision. Unlike Smart HPA [2], which improves scaling decisions, our work empirically characterizes the inherent limitations of CPU-based HPA under persistent WebSocket workloads. Recent reinforcement learning approaches further improve Kubernetes autoscaling decisions using adaptive policies, but still optimize QoS and resource efficiency rather than preserving persistent connection state [13].

3 Background: HPA Control Architecture

Before getting into the experiments, it helps to recap how the HPA control loop actually works. Nguyen et al. [23] cover this in detail for stateless workloads, and our own preliminary baselines reproduced their findings (Experiment A) - staircase scale-up, CPU overshoot right as a burst starts, and negligible benefit from scrape intervals below 30 seconds. That gave us confidence HPA behaves as expected in the stateless case before we moved on to the stateful one. Feedback control theory [10] is the formal basis for the proportional controller HPA implements, and elasticity in cloud computing is defined formally in [11].

Figure 1 shows the Kubernetes HPA control architecture we build on throughout this paper. The controller runs the following discrete-time proportional control law every 15 seconds, which is the default synchronization period:

$$desiredReplicas = \left\lceil currentReplicas \times \frac{currentMetricValue}{targetMetricValue} \right\rceil \tag{1}$$

This formula assumes `currentMetricValue` actually reflects how busy the workload is across every running replica. While production architectures often extend this mechanism using the Custom Metrics API and Prometheus Adapter, large-scale production Kubernetes deployments (e.g., CERN CMSWEB) similarly rely on extended observability and operational tooling around native Kubernetes autoscaling mechanisms [29, 3]. For CPU-based scaling of stateless workloads that assumption mostly holds up. It falls apart for persistent-connection workloads: an idle connection barely touches the CPU, yet it still represents a committed piece of session state, and disrupting it is very visible to whoever is on the other end. So the real experimental question is this: **what happens when the control design's core assumption - that the signal tracks demand - is simply not true?**

4 Empirical Evidence of HPA Failure for Stateful WebSocket Workloads

We ran a set of controlled experiments, each building on the last, designed to isolate one failure mode of CPU-based HPA at a time when it's applied to persistent WebSocket workloads. They're ordered from least severe (over-provisioning) to most severe (permanent connection destruction), and each one reuses the instrumentation and findings from the experiment before it.

All the WebSocket experiments share the same infrastructure and configuration, summarized in

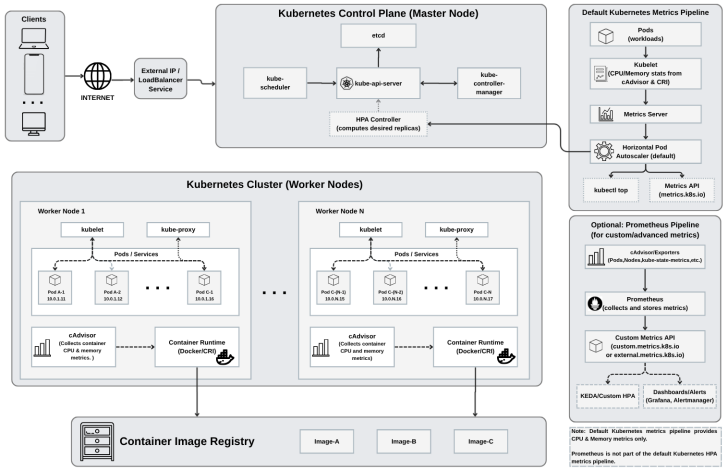

Fig. 1 Overview of the Kubernetes Horizontal Pod Autoscaler (HPA) control architecture used throughout this study.

Table 1. Everything runs on a 3-node Kind cluster [17, 22], with the Metrics Server [19] tuned for high-resolution scraping. The workload itself is a custom Python asyncio WebSocket server we call `ws-instrumented`. All the scripts, cluster configs, server code, and load generators used here are public.

Table 1 Summary of Experimental Architecture and Configuration

Component	Configuration
Kubernetes	v1.31.6
Kind	v0.25
Worker nodes	2
Metrics Server	15 s resolution
HPA target	CPU 60%
Prometheus scrape interval	15 s
WebSocket clients	800

Instrumented server. By "instrumented" we mean the server exposes a Prometheus gauge called `active_connections` and a counter called `new_connections_total` [26, 25]. Together these let us directly measure live session counts and reconnection rates using PromQL's `rate()` function. It's a small, minimally invasive change, but it gives us exactly the signal that CPU alone can't provide.

4.1 Experiment A: HPA Baseline Under Monotonic Correlated Load

Hypothesis. If load increases steadily and every connection actively generates CPU work (`CPU_WORK=1`), HPA should scale the deployment correctly. This gives us an empirical baseline to compare the later experiments against.

Experimental Setup. Eight hundred WebSocket clients connect gradually over 300 seconds. Every message the server receives triggers a bounded CPU spin loop, so active connection count and CPU utilization stay tightly coupled: CPU $\propto$ connections. HPA was set with `minReplicas=2`, `maxReplicas=10`, and `targetCPUUtilizationPercentage=60`.

Results. HPA scaled up smoothly from 2 to 5 replicas as CPU rose along with connection count. Once the load was removed, HPA waited out the 300-second stabilization window and then descended cleanly to `minReplicas`. There was no oscillation and no reconnection disruption at all, as Figure 2 shows. This is the best case, and it's only achievable when CPU actually tracks connection demand.

Takeaway. HPA does scale correctly when CPU tracks connections perfectly. The rest of the experiments look at what happens once that correlation breaks down.

4.2 Experiment B1: HPA Under Cyclic Load - Over-Provisioning and Recovery Asymmetry

Hypothesis. Under a realistic cyclic HIGH/LOW load pattern - the kind of thing you'd actually see with real connection activity - HPA should systematically over-provision and recover slower than it should.

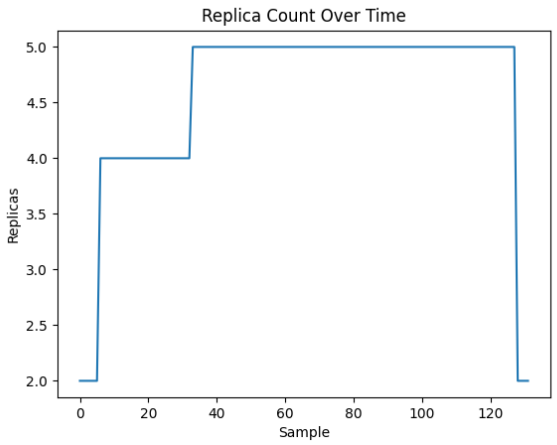

Fig. 2 Results from Experiment A establishing a correct-behavior baseline.

Experimental Setup. Load alternates between a HIGH phase (800 connections, active CPU-generating pinging, 60 s) and a LOW phase (connections held open but idle, no pinging, 30 s). During the LOW phase, all 800 client connections stay open - only the pinging stops, so CPU collapses toward zero while the session state itself is preserved. `maxReplicas` was set to 15, and we kept the default 300-second scale-down stabilization window to see how the factory settings would perform. We ran five consecutive HIGH/LOW cycles, each 90 seconds.

Results. HPA scaled up to an average peak of 10 replicas. Based on empirical profiling of our server configuration, each pod can sustainably handle approximately 100 active WebSocket connections before CPU utilization reaches the HPA target threshold. During the 60-second ramp-up, the average active connection count was approximately 285, yielding a connection-derived optimum of $\lceil 285/100 \rceil = 3$ replicas.

The resulting replica over-allocation is computed as:

$$\left(\frac{\text{Observed Replicas (10)}}{\text{Optimal Replicas (3)}} - 1 \right) \times 100 \approx 233\% \qquad (2)$$

What's driving this is the mismatch between the 90-second cycle and the 300-second window: each 30-second LOW phase ends well before the window has a chance to expire, so every HIGH phase starts back up near the previous cycle's ceiling. Replicas ratchet up to 10 and just stay there; getting back down to `minReplicas` after the final cycle took about 650 seconds. The multi-run average in Figure 3 confirms this holds across all 3 replications. Variance across replications remained low, indicating consistent behavior.

Interpretation. HPA simply can't tell the difference between a LOW period where connections are being held (CPU low, but capacity is still needed) and one where the pod is genuinely idle. HPA maintained 233% more replicas than required by the observed connection demand and required approximately 650 seconds to return to the minimum replica count. Although these additional replicas consumed cluster resources, the experiment quantifies replica over-allocation rather than direct infrastructure cost.

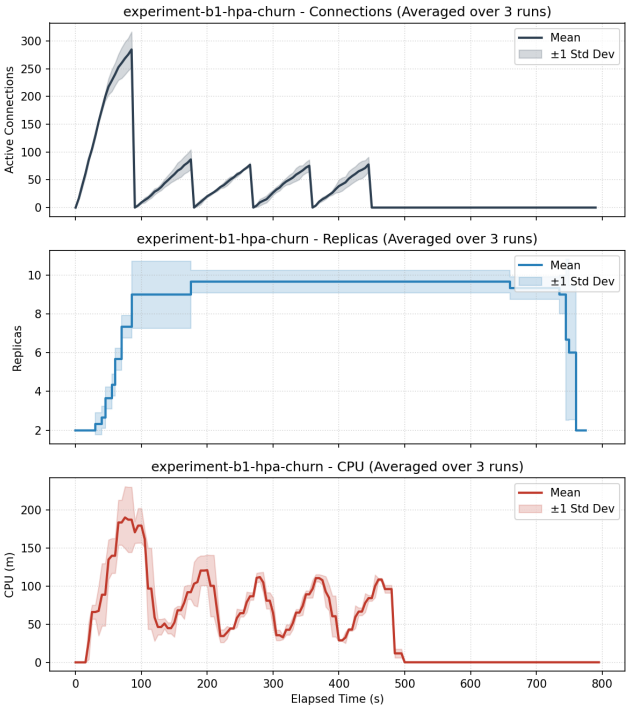

Fig. 3 Experiment B1: HPA replica over-allocation under cyclic load.

4.3 Experiment B2-Instrumented: Quantifying Reconnection Storm Intensity

Methodological note. Before this, we ran a preliminary, non-instrumented version (B2-Extended LOW) that confirmed connections were being severed during scale-down, but it gave us no quantitative data at all - no Prometheus metrics, no storm rate. It was mostly useful as motivation to add proper instrumentation, and we don't report it as a standalone result here.

Hypothesis. Each time HPA scales down and terminates a pod hosting active connections, we expect a measurable reconnection storm, along with real instability in the active connection count as clients race to reconnect all at once.

Experimental Setup. We instrumented the WebSocket server with two Prometheus metrics:

`active_connections` (a gauge for the current session count) and `new_connections_total` (a monotonically increasing counter that lets us compute rates via `rate()`). Prometheus scraped at a 15-second interval. HPA was configured with `stabilizationWindowSeconds: 0` for scale-up and `stabilizationWindowSeconds: 60` for scale-down. Setting scale-up to zero means each reconnection storm - which spikes CPU - immediately triggers a scale-up, so every cycle produces a complete, observable scale-up/scale-down sequence within our measurement window. Each cycle was a 60-second HIGH phase (800 pinging clients, `CPU_WORK=1`) followed by a 90-second LOW phase (connections idle, no pinging). We picked 90 seconds specifically because it's longer than the 60-second scale-down window, guaranteeing HPA executes at least one scale-down per cycle. We ran five complete HIGH/LOW cycles.

Results. Every single HPA scale-down event across all five cycles triggered a distinct, measurable reconnection storm. The storm rates were fairly consistent run to run, with peak observed rates falling between 406 and 512 conn/s across the replicates. Averaged across the three runs, the mean peak reconnection rate came out to 457 connections/second (left panel of Figure 4). At the same time, the active connection count got noticeably unstable during each storm. The mean peak reached 767 ($\pm$46), with individual runs ranging from 732 up to 819 (center panel of Figure 4). That spread - some runs falling below the 800-connection target and others overshooting it - is basically the signature of a reconnection storm: clients all disconnect at once and race to reconnect, producing a burst of competing attempts that the server can't resolve instantaneously.

Here's what's happening at the socket level: once a pod's 30-second termination grace period ends, it gets `SIGKILL`, which abruptly resets its TCP connections (TCP RST). Every client on that pod notices the closure at essentially the same moment and tries to reconnect right away. That burst of concurrent `connect()` calls puts transient pressure on the surviving pods' accept queues, so some clients end up connecting a little later than others, or fail and have to retry. That's why the peak connection count can overshoot 800 in one run (Run 2: 819) and fall short in another (Run 1: 732, Run 3: 750) - it comes down to exactly when HPA's `SIGKILL` lands relative to the cycle. The storm itself settles down within 15-30 seconds once the burst subsides and things stabilize again.

Figure 4 lays out the replica oscillation and its direct tie to the client-side session disruption we see in all three metrics.

Observation. Across the five cycles we measured, every HPA scale-down event lined up with a measurable reconnection storm. Each time replicas dropped, all sessions on the terminated pod got disconnected, and the storm's intensity scaled with how many connections that particular pod was holding. At a mean peak of 457 conn/s, with the active connection count varying widely across runs (767 $\pm$46), storms of this size mean a real service disruption for anything latency-sensitive - the session population just isn't stable or predictable during the reconnection burst.

4.4 Experiment B3: Permanent Connection Loss Under Non-Reconnecting Clients

Hypothesis. If WebSocket clients don't implement any reconnection logic - think batch data collectors, embedded IoT sensors, or long-running streaming sessions that treat a dropped connection as a terminal failure - then HPA scale-down events should cause a *permanent, irrecoverable* drop in the total session population.

Experimental Setup. We reconfigured the load generator with `reconnect: false`, so once a client's connection gets severed by a pod being terminated, it neither retries nor reconnects. The HPA policy matched B2-Instrumented: `stabilizationWindowSeconds: 0` for scale-up, `stabilizationWindowSeconds: 60` for scale-down. We kept CPU work on (`CPU_WORK=1`) so HPA would stay active. The IDLE phase ran for up to 240 seconds (`IDLE_MAX_DURATION=240`), giving HPA plenty of room to execute several sequential scale-down events.

Pod termination mechanics and the 30-second limbo It's worth walking through the exact Kubernetes pod termination sequence [16] here, since it matters for interpreting this experiment. When HPA reduces `spec.replicas`, this is what happens, in order: first, the target pod gets pulled from the Service's endpoint slice immediately, so no new connections get routed to it. Second, Kubernetes sends `SIGTERM` to the pod's main process and starts the `terminationGracePeriodSeconds` timer (30 s by default). Third, during that 30-second grace period, the existing TCP connections on the terminating pod stay open and functional - the OS kernel keeps them alive, not Kubernetes. Fourth, once

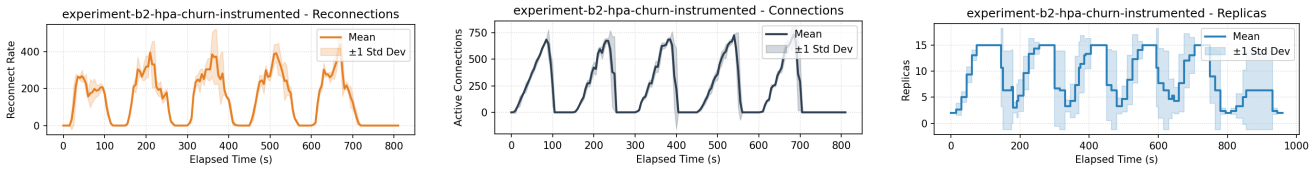

Fig. 4 Experiment B2-Instrumented: Reconnection storms triggered by HPA scale-down events.

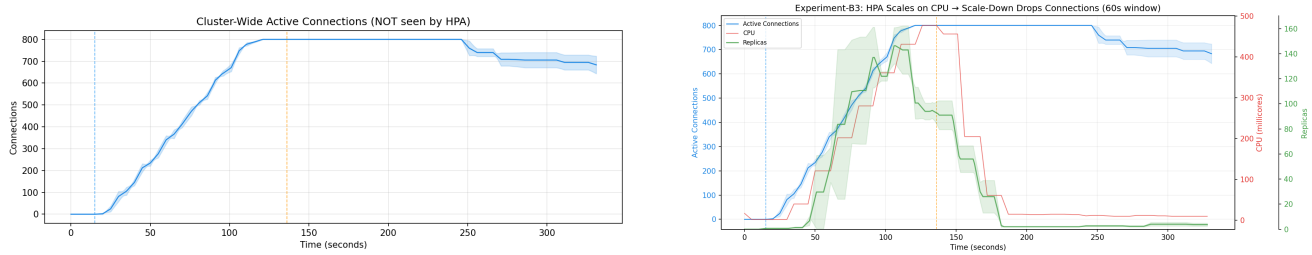

Fig. 5 Experiment B3: Staircase session destruction due to HPA scale-down.

`terminationGracePeriodSeconds` runs out, Kubernetes sends `SIGKILL`, which kills the process and forces the OS to reset every open TCP connection (TCP RST). This is why the connection drop in Figure 5 (left) lags roughly 30 seconds behind HPA's actual scale-down decision - the connection doesn't die the instant HPA acts, it dies 30 seconds later when `SIGKILL` finally fires. That delay doesn't soften the blow, though: the connection still dies, the client still gets a TCP RST, and with `reconnect: false` the session is gone for good.

Results. The active connection count (Figure 5, left) traces out a clear **staircase step-down** pattern. Each step lines up exactly with an HPA scale-down decision - when a pod gets terminated, its entire connection table is wiped out permanently. Every successive scale-down leaves fewer active connections than before, with nothing to bring them back. The representative run (Run 4 of the five-run replication; see Table 2) went 800 → 771 → 708 → 644, a 19.5% permanent loss.

The combined view (Figure 5, right) shows how all three signals relate over time: connections go idle and stop generating CPU load, HPA sees near-zero CPU and decides to scale down, and the terminated pod then permanently wipes out every connection it was hosting. None of this can be fixed just by tweaking a configuration parameter - it follows directly from how HPA observes, or rather fails to observe, connection state in the first place.

Summary. Under the configuration we tested, HPA's CPU-based signal leads it to scale down in ways that destroy idle-but-connected sessions. Parameter tuning alone doesn't fix this. Recent adaptive threshold approaches improve scaling decisions under varying workloads [24]; however, they still depend on resource-utilization metrics and therefore do not address connection affinity. This is exactly why future work investigates a connection-aware autoscaling mechanism.

Replication and Statistical Validation To make sure this staircase destruction pattern isn't just a one-off artifact, we replicated Experiment B3 five times under identical conditions: same Kind cluster setup, same HPA policy (`stabilizationWindowSeconds: 60` for scale-down), 800 non-reconnecting clients, and the same two-phase protocol (120-second CONNECT phase followed by a 240-second IDLE phase). Each run used a freshly provisioned cluster so there was no state carrying over between replications. Table 2 lays out the per-run connection counts and loss percentages, and Table 3 shows the active connection count at each observed staircase step for every run.

Table 2 Statistical replication of Experiment B3 (5 runs).

Run	Peak Conn.	Final Conn.	Loss (%)
Run 1	800	637	20.4%
Run 2	800	652	18.5%
Run 3	800	658	17.8%
Run 4	800	644	19.5%
Run 5	800	656	18.0%
Mean ± SD	**800 ± 0**	**649.4 ± 8.8**	**18.8% ± 1.1%**

A few things about these results make us fairly confident the staircase pattern is a deterministic consequence of how HPA is built, rather than something random:

Table 3 Active connection count at each observed staircase step during the IDLE phase for all five B3 replications.

Run	Step 0	Step 1	Step 2	Final
Run 1	800	740	-	637
Run 2	800	718	686	652
Run 3	800	761	668	658
Run 4	800	771	708	644
Run 5	800	741	735	656

1. **The destruction magnitude barely moves between runs.** Final connection counts across all five runs sit in a narrow 637-658 band, giving a standard deviation of just 8.8 connections (1.1% in loss terms). That kind of low variance tells us the number of connections lost per scale-down event comes down to how connections happen to be distributed across pods, not random chance.
2. **The staircase shape itself never changes.** Every single run goes through the same three phases: a monotonic climb from 0 to 800 during the CONNECT phase, a short plateau at 800 while CPU is still elevated, and then a staircase descent once CPU drops and HPA starts scaling down. How many steps you see (2 or 3) depends on exactly when HPA's evaluation cycle lines up with the start of the IDLE phase, but the overall shape doesn't change.
3. **Loss per step varies, but for a clear reason.** Individual scale-down steps destroy anywhere from 6 to 163 connections, which just reflects however many connections the terminated pod happened to be holding. That variation comes from Kubernetes' round-robin Service routing spreading connections unevenly across pods, not from any randomness in the HPA control loop itself.

4.5 Summary Across Experiments

Table 4 pulls together the results from all four experiments.

Table 4 Summary of results across the four experiments.

Experiment	Condition	HPA Behavior	Key Quantitative Finding
A (Baseline)	Monotonic load; CPU ∝ connections	Correct proportional scaling	Works only under idealized signal correlation
B1 (Cyclic)	Alternating HIGH/LOW activity	Reaches 10 replicas rapidly	233% replica over-allocation; 650 s to recover
B2-Inst. (Measured)	Cyclic + Prometheus instrumentation	Reconnection storm per scale-down	Mean peak 457 conn/s; connection instability (767 ±46 peak, range 732–819)
B3 (No-reconnect)	Idle connections; no client retry	Permanent, staircase connection loss	800→649 mean final (18.8% loss); zero recovery

5 Discussion

5.1 CPU as a Scaling Signal for Stateful Workloads

We can capture what's going wrong across Experiments B1-B3 with two quantities. Call $M(t)$ the *scaling metric* HPA is watching at time t - under the standard setup, that's average CPU utilization across all running replicas. Call $D(t)$ the actual *workload demand* at time t: the number of active connections $C(t)$ whose session state needs to stay alive on a running pod. HPA's control law (Equation 1) computes the desired replica count purely from $M(t)$, and that only gives correct scaling if $M(t)$ actually tracks $D(t)$:

$$M(t) \propto D(t) \quad \Longrightarrow \quad \mathit{desiredReplicas}(t) \propto D(t) \tag{3}$$

For stateless HTTP workloads, where each request grabs a bounded chunk of CPU and lets go of it when it's done, this proportionality basically holds in practice - Experiment A backs that up.

For persistent connections that go through idle periods, Equation 3 stops holding. While connections are actively pinging, they generate CPU work and $M(t) \propto D(t)$ still holds. But during idle stretches, the connections are still open - demand $D(t) = C(t) > 0$ hasn't gone away - while CPU utilization drops toward zero anyway:

$$M(t) \to 0 \quad \text{while} \quad D(t) = C(t) > 0 \tag{4}$$

Plug that into Equation 1 and *desiredReplicas* gets pushed down toward `minReplicas`, so HPA starts scaling down pods that are still hosting live connections. Put Equations 3 and 4 side by side and you get a picture where the scaling signal and the actual demand move in opposite directions: the metric falls exactly when demand is at its highest.

What we saw in Experiments B1, B2-Instrumented, and B3 lines up with this account. In B3 specifically, CPU utilization fell toward zero because connections had gone idle, and that drop happened before every scale-down decision. Variance across replications remained low, indicating consistent behavior. Put all three experiments together and they support the idea that CPU utilization, at least in the setups we tested, just isn't a reliable stand-in for WebSocket session demand - and sometimes moves the exact opposite way. Proper metric selection is critical for stability, as demonstrated for containerized applications under varying workloads [5], and is further complicated by Kubernetes' inherent resource management and scheduling overheads [21].

A connection-aware controller explored in subsequent work gives us a useful comparison point: a controller running at `CPU_WORK=0` for 570 seconds holds steady at 8 replicas using nothing but a connection-count signal. That fits with the idea that the problem here comes down to which signal HPA is watching, not the proportional control law from Equation 1 itself - give the same control law a signal that actually tracks $D(t)$, and it does exactly what you'd expect. Recent intelligent autoscalers improve scaling decisions [13], but this does not bypass the fundamental requirement of tracking connection state. Large-scale production deployments have likewise adopted Kubernetes custom metrics infrastructures to improve observability and operational stability [29]. Recent research has focused on jointly optimizing pod- and infrastructure-level elasticity for better resource utilization [8], whereas our work focuses on preserving application correctness under persistent connection workloads.

5.2 The Stabilization Window and Idle Connections

One obvious fix people suggest is just making HPA's stabilization window bigger so it outlasts the expected idle period. The tuning study in Section 5.4 shows this doesn't help much, and the reason has more to do with what the window is actually buffering than with how big it is.

The stabilization window works on a time series of *desiredReplicas* values, and each one comes from a CPU observation. If CPU stays near zero for the whole idle period, every value going into that buffer is `minReplicas`. Since the stabilized output is just the maximum over the buffer, you end up at `minReplicas` no matter how the window is sized - a longer window just delays the outcome, it doesn't change it.

If instead the window buffered *connection-derived* desired counts - a rolling high-water mark of recent capacity needs computed straight from the connection metric - it would be buffering something completely different from what HPA currently produces. That's a change to what the mechanism operates on, not just its parameters, which is explored in subsequent work.

5.3 Pod Termination Timing

Kubernetes' pod termination sequence leaves a 30-second gap between `SIGTERM` and `SIGKILL` during which connections on the terminating pod are still open at the OS level. Some people treat this as a chance for graceful draining via `preStop` hooks. But WebSocket sessions typically run for minutes to hours, so a 30-second window is tiny by comparison, and it's not much use for connection migration or draining in this context.

A better place to intervene might be earlier in the sequence - right where the scale-down decision itself gets made, before termination even starts. If you suppress or delay that decision while a pod still has live sessions on it, you fix the problem at the source rather than trying to patch things during the termination window. That's the direction we take in subsequent work.

5.4 HPA Stabilization Window Tuning Study

A natural question is whether aggressively tuning `scaleDown.stabilizationWindowSeconds` to outlast the expected idle gap could fix HPA's destructive behavior during idle phases. We tested this directly with a systematic tuning study across four window sizes - 30, 60, 120, and 300 seconds - each tested under two different gap conditions.

Experimental Methodology. For each window size $W \in \{30, 60, 120, 300\}$ seconds, we ran the B3 workload protocol (800 non-reconnecting clients, 120-second CONNECT phase with `CPU_WORK=1`) followed by an idle gap of length G. We tested two conditions per window: $G < W$ ("below," where the gap is shorter than the window and connections should be protected), and $G > W$ ("above," where the gap outlasts the window and HPA is expected to scale down). We picked $G_{\text{below}} = W - 15$ s and $G_{\text{above}} = W \times 1.5$ to $W + 60$ s, giving margin on both sides of the boundary. All eight configurations (4 windows $\times$ 2 gap conditions) were replicated three times each on fresh Kind clusters, for 24 sub-experiments in total.

Results. Table 5 shows the aggregated results. The pattern couldn't be clearer: when the idle gap sits below the stabilization window, connections survive every time (0% loss across all 12 below-window runs). Once the gap exceeds the window, connections get destroyed in every configuration we tested, with mean loss rates running from 5.2% at a 30-second window up to 33.2% at a 300-second window.

Analysis. These results tell us two things about HPA's stabilization mechanism. First, the window works more like a delay than a protection: it pushes the scale-down decision back by W seconds without

Table 5 HPA stabilization window tuning study results (3 replications per configuration).

Window	Gap	Relation	Loss (%)	Survived
30s	15s	below	0.0 ± 0.0	3/3
30s	90s	above	5.2 ± 6.2	2/3
60s	45s	below	0.0 ± 0.0	3/3
60s	120s	above	10.2 ± 7.2	2/3
120s	105s	below	0.0 ± 0.3	3/3
120s	180s	above	10.4 ± 5.6	1/3
300s	285s	below	0.0 ± 0.0	3/3
300s	360s	above	33.2 ± 28.0	1/3

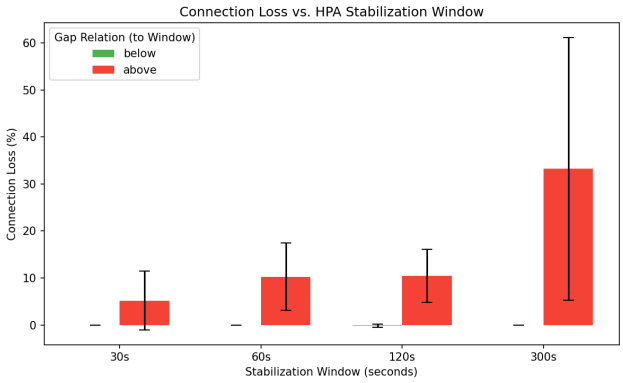

Fig. 6 Connection loss percentage relative to the HPA stabilization window and gap duration.

ever actually preventing it. If CPU stays near zero for the whole window, every entry in the recommendation buffer converges to `minReplicas`, and the stabilized output - the max over the window - ends up at `minReplicas` too.

Second, the variance in the "above" condition gets a lot bigger at larger window sizes (SD = 28.0% at $W = 300$s versus SD = 6.2% at $W = 30$s), which is probably tied to how HPA's 15-second evaluation cycle lines up with the window boundary. When the idle gap only barely exceeds the window, whether a scale-down event actually lands inside the measurement period depends on that alignment - which adds variance that has nothing to do with the window's protective effect.

Across every window size we tested, we never found a configuration where a finite stabilization window actually prevented connection loss once the idle gap outlasted the window. That fits with the idea that the window is a buffer on CPU-derived recommendations, not a mechanism that's actually sensitive to connection state.

6 Threats to Validity and Limitations

1. **Evaluation environment.** We ran every experiment on a single-machine Kind cluster. That gives us full reproducibility and tight control, but it doesn't capture cross-node scheduling latency, network partitions, or the metric-collection noise you'd see on a real multi-node cloud cluster. The direction of our results should hold up; the exact timing numbers (pod startup lag, HPA sync precision) may look different on production infrastructure. Although our experiments were conducted on a Kind cluster, similar Kubernetes monitoring architectures have been deployed successfully in large-scale production environments such as CERN CMSWEB [29, 3]. Validation of these dynamics on managed Kubernetes offerings (such as GKE, EKS, or AKS) is left for future work.
2. **Workload generalizability.** Our load generator uses a synthetic WebSocket workload with a linear connection ramp and deterministic silence gaps. Real-world connection arrival tends to be burstier, and session durations vary a lot more. Under stationary random load (Poisson arrivals), we'd expect the same proportional failure mechanisms to apply; only the peak connection count and storm rate we observed would likely change.
3. **Instrumentation dependency.** The B2-Instrumented results depend on Prometheus scraping metrics every 15 seconds. A coarser scrape interval would affect how precisely we can measure the storm rate, but it wouldn't change the underlying causal finding that scale-down events trigger reconnection cascades.
4. **Protocol generalizability.** The evaluation focuses on CPU-based HPA and WebSocket workloads. Other stateful protocols such as MQTT or gRPC streaming may exhibit different quantitative behavior although similar qualitative limitations are expected.
5. **Single reconnect=false regime.** Experiment B3 uses a uniform `reconnect: false` policy across the board. Real deployments usually have a mix of reconnecting and non-reconnecting clients, so the staircase destruction rate and the final session population would likely look different under a mixed client policy.

7 Conclusion

This paper looked at how Kubernetes HPA behaves under stateful WebSocket workloads through four controlled experiments, each one building on the last.

Experiment A showed HPA scales correctly when CPU tracks connections perfectly - a condition that's rarely true outside a lab. Experiment B1 put HPA through a realistic HIGH/LOW activity cycle and produced 233% replica over-allocation (10 replicas instead of the correct 3), hit `maxReplicas` within 99 seconds, and took roughly 650 seconds to recover. Experiment B2-Instrumented added Prometheus instrumentation and directly measured what scale-down actually costs: a peak reconnection rate of 457 connections/second, and real instability in the active connection count (mean peak of 767 ±46, ranging from 732 to 819 against an 800-client target), both lining up exactly with HPA's scale-down events. Experiment B3, with reconnection turned off, produced a staircase pattern of permanent connection loss - across five replications, the session population dropped from 800 to a mean of 649 (±1.1%, or 18.8% loss), with nothing bringing it back.

Put together, these results point to a limitation that has more to do with how HPA is designed than with any one configuration choice. CPU utilization, at least in the setups we tested, doesn't work as a reliable stand-in for connection-based demand, and the tuning study in Section 5.4 shows that adjusting the stabilization window doesn't really change that once the idle period outlasts the window. The window buffers CPU-derived recommendations, not connection counts - the problem is what gets measured, not how long the measurement gets held onto.

One takeaway from all this is that autoscaling for stateful, persistent-connection workloads probably needs two things working together: a scaling signal built directly from connection counts, since that reflects session state honestly, and scale-down logic that actually checks for live sessions before it terminates a pod. These findings give future connection-aware autoscaling work a quantitative baseline to build on, and they make a decent case for rethinking CPU-centric autoscaling wherever persistent, cloud-native applications are involved.

Declarations

- **Data Availability** The source code, Kubernetes manifests, Prometheus dashboards, experimental scripts, datasets, and plotting utilities used in this study are publicly available at: `https://github.com/MistaHolmes/ws-k8-controller`.
- **AI Declaration** During the preparation of this manuscript, the authors used generative AI tools, including Claude and ChatGPT, to assist with language refinement, grammar correction, manuscript organization, and editorial improvements. The study conception, methodology, software implementation, experimental design, data collection, analysis, interpretation of results, and scientific conclusions were developed, verified, and approved by the authors. The authors carefully reviewed and edited all AI-assisted content and take full responsibility for the final manuscript.

References

1. Toward optimal load prediction and customizable autoscaling scheme for kubernetes. Mathematics **11**(12), 2675 (2023). DOI 10.3390/math11122675
2. Ahmad, H., Treude, C., Wagner, M., Szabo, C.: Smart HPA: A resource-efficient horizontal pod auto-scaler for microservice architectures. In: Proceedings of the IEEE International Conference on Software Architecture (ICSA), pp. 46–57. IEEE (2024). DOI 10.1109/ICSA59870.2024.00013
3. Ali, A., Imran, M., Kuznetsov, V., Trigazis, S., Pervaiz, A., Pfeiffer, A., Mascheroni, M.: Implementation of new security features in cmsweb kubernetes cluster at cern. Cluster Computing (2024). DOI 10.1007/s10586-024-04626-6
4. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J.: Borg, omega, and kubernetes **14**(1), 70–93 (2016). DOI 10.1145/2898442.2898444
5. Casalicchio, E.: A study on performance measures for auto-scaling cpu-intensive containerized applications. Cluster Computing **22**, 995–1006 (2019). DOI 10.1007/s10586-018-02890-1
6. Casalicchio, E., Perciballi, V.: Auto-scaling of containers: the impact of relative and absolute metrics. In: 2017 IEEE 2nd International Workshops on Foundations and Applications of Self-* Systems (FAS*W), pp. 207–214. IEEE, New York, NY, USA (2017). DOI 10.1109/FAS-W.2017.149
7. Dickel, H., Podolskiy, V., Gerndt, M.: Evaluation of autoscaling metrics for (stateful) IoT gateways. In: 2019 IEEE 12th Conference on Service-Oriented Computing and Applications (SOCA), pp. 17–24. IEEE, New York, NY, USA (2019). DOI 10.1109/SOCA.2019.00011

8. Entrialgo, J., García, M., García, J., López, J.M.: Joint autoscaling of containers and virtual machines for cost optimization in container clusters. Journal of Grid Computing **22**, 17 (2024). DOI 10.1007/s10723-023-09732-4
9. Fette, I., Melnikov, A.: The WebSocket Protocol. RFC 6455, IETF (2011). DOI 10.17487/RFC6455
10. Hellerstein, J.L., Diao, Y., Parekh, S., Tilbury, D.M.: Feedback Control of Computing Systems. John Wiley & Sons, Hoboken, NJ, USA (2004)
11. Herbst, N.R., Kounev, S., Reussner, R.: Elasticity in cloud computing: What it is, and what it is not. In: Proceedings of the 10th International Conference on Autonomic Computing (ICAC), pp. 23–27. USENIX Association, Berkeley, CA, USA (2013)
12. Hightower, K., Burns, B., Beda, J.: Kubernetes: Up and Running. O'Reilly Media, Sebastopol, CA (2017)
13. Khaleq, A.A., Ra, I.: Intelligent microservices autoscaling module using reinforcement learning. Cluster Computing **26**(5), 2789–2800 (2023). DOI 10.1007/s10586-023-03999-8
14. Kubernetes Authors: Container lifecycle hooks - kubernetes documentation. `https://kubernetes.io/docs/concepts/containers/container-lifecycle-hooks/` (2024). Accessed: July 2026
15. Kubernetes Authors: Horizontal pod autoscaling - kubernetes documentation. `https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/` (2024). Accessed: July 2026
16. Kubernetes Authors: Pod lifecycle - kubernetes documentation. `https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/` (2024). Accessed: July 2026
17. Kubernetes SIGs: kind - kubernetes in docker. `https://kind.sigs.k8s.io/` (2024). Accessed: July 2026
18. Kubernetes SIGs: Kubernetes custom metrics adapter for Prometheus. `https://github.com/kubernetes-sigs/prometheus-adapter` (2024). Accessed: July 2026
19. Kubernetes SIGs: Metrics server. `https://github.com/kubernetes-sigs/metrics-server` (2024). Accessed: July 2026
20. Lorido-Botrán, T., Miguel-Alonso, J., Lozano, J.A.: A review of auto-scaling techniques for elastic applications in cloud environments. Journal of Grid Computing **12**(4), 559–592 (2014). DOI 10.1007/s10723-014-9314-7
21. Medel, V., Tolosana-Calasanz, R., Ángel Bañares, J., Rana, O.F.: Characterising resource management performance in kubernetes. Future Generation Computer Systems (2024). Accepted / published in 2024
22. Merkel, D.: Docker: Lightweight linux containers for consistent development and deployment **2014**(239) (2014)
23. Nguyen, T.T., Yeom, Y., Kim, T., Park, D.H., Kim, S.: Horizontal Pod Autoscaler in Kubernetes for elastic container orchestration. Sensors **20**(16), 4621 (2020). DOI 10.3390/s20164621
24. Pozdniakova, O., Mažeika, D., Cholomskis, A.: Sla-adaptive threshold adjustment for a kubernetes Horizontal Pod Autoscaler. Electronics **13**(7), 1242 (2024). DOI 10.3390/electronics13071242
25. Prometheus Authors: Prometheus documentation. `https://prometheus.io/docs/introduction/overview/` (2024). Accessed: July 2026
26. Prometheus Authors: Prometheus python client. `https://github.com/prometheus/client_python` (2024). Accessed: July 2026
27. Qu, C., Calheiros, R.N., Buyya, R.: Auto-scaling web applications in clouds: A taxonomy and survey. ACM Computing Surveys **51**(4), 1–33 (2018). DOI 10.1145/3148149
28. Rossi, F., Nardelli, M., Cardellini, V.: Horizontal and vertical scaling of container-based applications using reinforcement learning. In: 2019 IEEE 12th International Conference on Cloud Computing (CLOUD), pp. 329–338. IEEE, New York, NY, USA (2019). DOI 10.1109/CLOUD.2019.00061
29. Tedeschi, M., Imran, M., Kuznetsov, V., Trendafilov, T., Ciangottini, D., Legger, F., Magini, N.: Migration of cmsweb cluster at cern to kubernetes: A comprehensive study. Cluster Computing (2021). DOI 10.1007/s10586-021-03325-0
30. Zhou, Z., Zhang, C., Ma, L., et al.: AHPA: Adaptive horizontal pod autoscaling systems on alibaba cloud container service for kubernetes. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 15621–15629 (2024). DOI 10.1609/aaai.v37i13.26852